

Summary of Code

- Approach

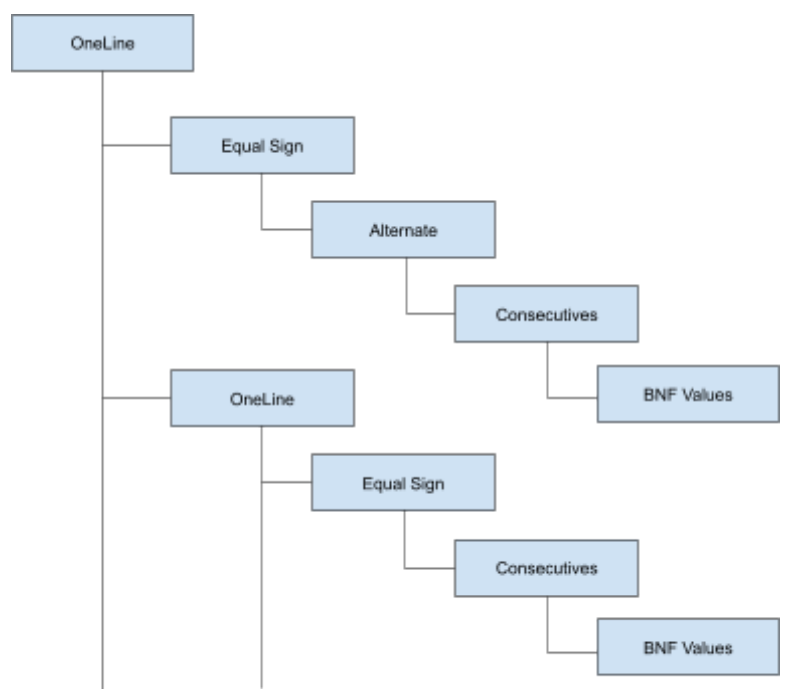
The code starts off with first parsing the building blocks that will be represented in the ADT, meaning the terminal and non terminal, or macros. Afterwards, it considers the modifiers applied to any of them. Then it checks how the elements of the rule are connected, with consecutive elements having a higher priority than rules connected with an alternate symbol. After parsing the rules, it then connects it with the LHS of the equal sign by using the EqualSign Type. Multiple of these EqualSigns can be connected by using the OneLine. The parsing ensures that there is whitespace between the components and ignores any leading or trailing whitespaces too.

After parsing, the code generates a Haskell Code from the ADT by traversing through it recursively. This will generate the types of the parser and the code of the actual parser themselves. The bnf is validated before the code is generated and is also checked recursively and iteratively (meaning that every time it deletes a rule with issues, it's repeated from the start the checking until every rule passes or every rule is deleted), and any potential issues in the BNF grammar is reported.

- Structure

If there are multiple lines of grammar, then those lines will be connected using OneLine where the right part corresponds to the rules that are seen later down while the ones on the left are the ones that came earlier.

The EqualSign represents a single line of rule where it connects the LHS of the rule which is the name of the rule and the right side of the rule which is how the rule is defined. It then connects multiple elements of the rule in succession using the Consecutive type and multiple consecutive types are connected with each other using the Alternate type if the rule has multiple alternatives to its definition.



Structure of ADT parse tree

Three Different Chains

For the chain functions, there are three distinct types, one for combining multiple lines, one for handling each line, and the last for handling the rules. By making three distinct chain functions, it makes it easier to consider how the behaviour of the parsers should be in each level, especially in the Equal-Sign level where the parameters references needs to be checked as the rule is being parsed or failing when the parsers fail to parse the input fully. These forms of different validation of the input is made very difficult if a single generic chain function is used.

These chains also allow multiple types to be combined together. So the result can be a single final ADT which was formed from parsing the input string.

A lot of Types

The reason for the many defined data types in the ADT to handle the different aspects of the BNF grammar, is because it will simplify the implementation of the next tasks, such as B and E. This is because of Haskell's pattern matching feature which allows functions to consider fewer cases. Since the types have properties tied with the BNF grammar, the code can exploit these certainties which can guarantee generating or validating much easier.

For example, having a OneLine constructor means that there are at least two lines of grammar. That the right part of the OneLine will always be a rule that can be fed into another function to generate the corresponding type and haskell code for it. The left part might be another OneLine constructor or a rule.

This structure reduces the amount of case handling required and makes the code easier to extend and also guarantees syntactic correctness since invalid grammars cannot be formed.

Common patterns to Functions

By converting common code patterns into its own function, like parsing modifiers, it makes the code reusable and avoids repeated implementation. It also makes the code more functional in its design as it composes smaller functions together to create one larger function that is more complex.

Even some functions can be re-used and be repurposed for something else, like the error report being used as a way to detect errors and delete rules with issues in the validation for generateHaskellCode.

Recursion for Task B and E

The reason for using recursion for these tasks instead of bind is because the previous values don't really have an effect on the current value and due to its recursive structure. While what the LHS of the rule and the number of fields in that rule does affect how the types and parser are generated, this can also be done intuitively with recursions even though it is more verbose and requires much more lines.

Uses of Bind (>>), Applicatives (<*>), Functors (<\$>)

The reason bind was used in the code was because the parsers relied on the result of previous parsers. After parsing a portion of the string, for example a single character, the remaining unparsed string is passed onto the next parser. This chain continues until an ADT value is produced and this is how the ADT parsers in the code are constructed.

The reason functors were used is to map the result from the parser into the type constructor and this allows the parsed values to be automatically wrapped in the corresponding ADT constructor. The reason applicatives were used is because it can create parsers by combining smaller parsers without needing to chain results, in here it's also used to enforce the structure of the input string (like a closing square bracket in macros) and this ensures type correctness since the output of the parser aligns with the corresponding data constructor.

This is how the ADT parsers in the code are constructed, where the result of the parsers are chained together using bind and the parsed value is mapped into the type constructors using functors, and using the applicative to validate the structure of the input string so that the correct type constructor is used.